Name: Vansh Jain
Roll No: 124A8036
Class/Div: G-2
Date of Submission: 13/03/2026

Experiment No.: 08

Title: To implement Memory Management using Multiprogramming with Fixed Tasks (MFT) and Multiprogramming with Variable Tasks (MVT) to demonstrate memory allocation techniques in operating systems.

Objective:
Using the CPU-OS simulator analyze and synthesize the following:
a. Understand memory allocation using Multiprogramming with Fixed Tasks (MFT).
b. Implement Multiprogramming with Variable Tasks (MVT) for dynamic memory allocation.
c. Compare fixed partitioning and dynamic partitioning techniques.

Software: Ubuntu

Theory: Memory management is one of the most important responsibilities of an operating system. It controls how memory is allocated to different processes and ensures efficient utilization of available memory resources. The operating system must allocate memory to processes in such a way that multiple programs can execute simultaneously without interfering with each other.
One of the early techniques used for memory management is Multiprogramming with Fixed Tasks (MFT). In this method, the main memory is divided into a fixed number of partitions before program execution begins. Each partition can contain only one process. If the process size is smaller than the partition size, the remaining space remains unused, which leads to internal fragmentation.
Another technique is Multiprogramming with Variable Tasks (MVT). In this method, memory partitions are created dynamically depending on the memory requirements of processes. Instead of fixed partitions, memory is allocated as needed. This improves memory utilization compared to MFT. However, it may lead to external fragmentation, where small free memory blocks are scattered throughout memory.
These techniques help in understanding how operating systems allocate memory and manage multiple processes efficiently.

Algorithm:

Part A: MFT (Multiprogramming with Fixed Tasks)

1. Input the number of memory partitions.
2. Enter the size of each memory partition.
3. Input the number of processes.
4. Enter the memory requirement of each process.
5. Initialize the allocation array with -1 (indicating no allocation).
6. For each process, check partitions sequentially.
7. If the partition size is greater than or equal to the process size and not already allocated:
    - Allocate the process to that partition.
8. Display the partition number allocated to each process.
9. If no suitable partition is found, mark the process as not allocated.

Part B: MVT (Multiprogramming with Variable Tasks)

1. Input the total available memory.
2. Input the number of processes.
3. Enter the memory requirement of each process.
4. Initialize remaining memory equal to total memory.
5. For each process:
    - Check if required memory is less than or equal to remaining memory.
6. If yes:
    - Allocate memory to the process.
    - Reduce remaining memory.
7. Otherwise:
    - Display that memory cannot be allocated.
8. Display the remaining memory after each allocation.

Output/Result:

Part A:

```
siesgst@aids: ~/naitik/oslab

siesgst@aids:~$ cd ~
siesgst@aids:~$ cd naitik
siesgst@aids:~/naitik$ cd oslab
siesgst@aids:~/naitik/oslab$ vi mft.c
siesgst@aids:~/naitik/oslab$ gcc mft.c -o mft
siesgst@aids:~/naitik/oslab$ ./mft
Enter number of partitions: 3
Enter partition sizes:
100
500
200
Enter number of processes: 4
Enter process sizes:
120
80
300
150

Process No      Process Size    Partition No
1               120             2
2               80              1
3               300             Not Allocated
4               150             3
siesgst@aids:~/naitik/oslab$
```

siesgst@aids: ~/naitik/osl

```c
#include <stdio.h>

int main() {

    int partitionSize[10], processSize[10];
    int allocation[10];
    int nPartitions, nProcesses;
    int i, j;

    printf("Enter number of partitions: ");
    scanf("%d", &nPartitions);

    printf("Enter partition sizes:\n");
    for(i = 0; i < nPartitions; i++)
        scanf("%d", &partitionSize[i]);

    printf("Enter number of processes: ");
    scanf("%d", &nProcesses);

    printf("Enter process sizes:\n");
    for(i = 0; i < nProcesses; i++)
        scanf("%d", &processSize[i]);

    for(i = 0; i < nProcesses; i++)
        allocation[i] = -1;

    for(i = 0; i < nProcesses; i++) {
        for(j = 0; j < nPartitions; j++) {

            if(partitionSize[j] >= processSize[i]) {

                allocation[i] = j;
                partitionSize[j] = 0;
                break;

            }
        }
    }

    printf("\nProcess No\tProcess Size\tPartition No\n");
-- INSERT --
```

```
    printf("\nProcess No\tProcess Size\tPartition No\n");

    for(i = 0; i < nProcesses; i++) {

        printf("%d\t\t%d\t\t", i+1, processSize[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }

    return 0;
}
-- INSERT --
```

Part B:

```
siesgst@aids:~/naitik/oslab$ vi mvt.c
siesgst@aids:~/naitik/oslab$ gcc mvt.c -o mvt
siesgst@aids:~/naitik/oslab$ ./mvt
Enter total memory: 1000
Enter number of processes: 4
Enter memory required for process 1: 200
Memory allocated for process 1
Remaining memory = 800
Enter memory required for process 2: 300
Memory allocated for process 2
Remaining memory = 500
Enter memory required for process 3: 400
Memory allocated for process 3
Remaining memory = 100
Enter memory required for process 4: 150
Memory not allocated for process 4
Remaining memory = 100
siesgst@aids:~/naitik/oslab$
```

siesgst@aids: ~/naitik/oslab

```c
#include <stdio.h>

int main() {

    int totalMemory, remainingMemory;
    int processSize[10];
    int nProcesses, i;

    printf("Enter total memory: ");
    scanf("%d", &totalMemory);

    remainingMemory = totalMemory;

    printf("Enter number of processes: ");
    scanf("%d", &nProcesses);

    for(i = 0; i < nProcesses; i++) {

        printf("Enter memory required for process %d: ", i+1);
        scanf("%d", &processSize[i]);

        if(processSize[i] <= remainingMemory) {

            printf("Memory allocated for process %d\n", i+1);
            remainingMemory -= processSize[i];

        }
        else {

            printf("Memory not allocated for process %d\n", i+1);

        }

        printf("Remaining memory = %d\n", remainingMemory);
    }

    return 0;
}
~
~
-- INSERT --
```

Conclusion: The experiment successfully demonstrated memory allocation techniques using MFT and MVT methods. MFT allocates processes to fixed partitions but may cause internal fragmentation due to unused memory space. MVT dynamically allocates memory based on process requirements, improving memory utilization but potentially causing external fragmentation. These methods help in understanding how operating systems efficiently manage memory for multiple processes.